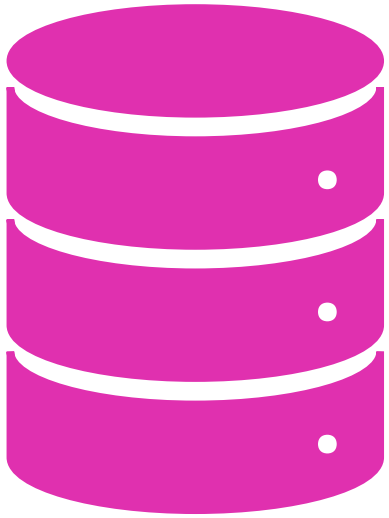
MONGODB

SECTION 2

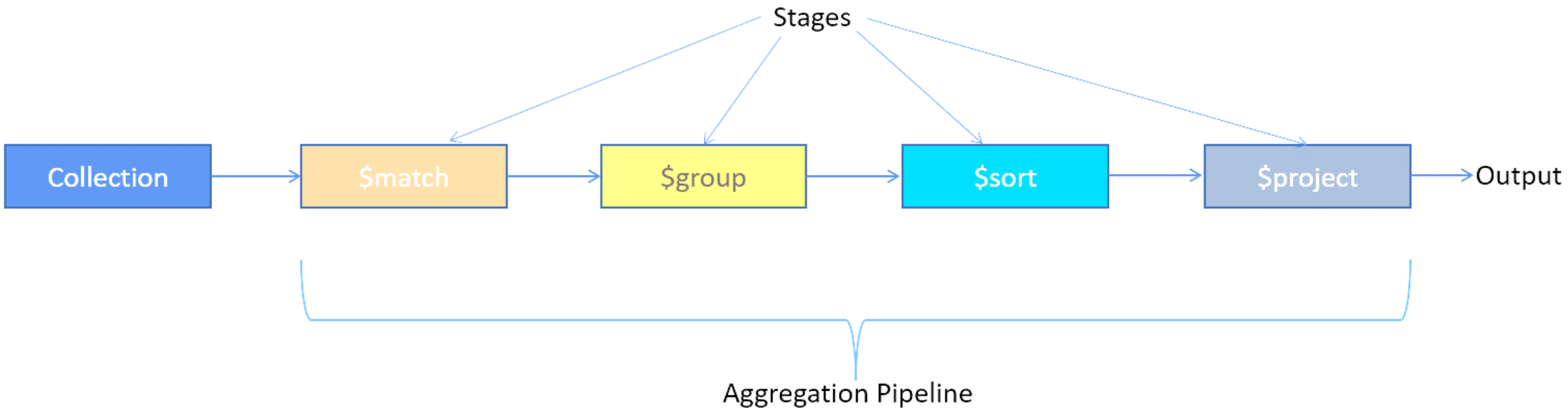
ENG. ASMAA FATHY



AGGREGATION PIPELINES

AGGREGATION PIPELINES

- What are Aggregation Pipelines? Aggregation Pipelines are a method of performing a series of operations to process data step by step, where each operation is performed on the data coming out of the previous operation. This is similar to the concept of a pipeline, where data is sent through a series of processing stages.



FIRST, CREATE A DATABASE

```
use storeDB; // Create and use the storeDB database

db.createCollection('sales'); // Create a collection named 'sales'

// insert some data into 'sales' collection
db.sales.insertMany([
{ store: 'Store_A', product: 'Laptop', quantity: 10, price: 1500, salesValue: 15000, region: 'East', orderDate: ISODate('2023-01-01T10:00:00Z') },
{ store: 'Store_B', product: 'Phone', quantity: 5, price: 700, salesValue: 3500, region: 'West', orderDate: ISODate('2023-01-02T11:00:00Z') },
{ store: 'Store_C', product: 'Tablet', quantity: 7, price: 300, salesValue: 2100, region: 'East', orderDate: ISODate('2023-01-03T12:00:00Z') },
{ store: 'Store_A', product: 'Monitor', quantity: 4, price: 200, salesValue: 800, region: 'South', orderDate: ISODate('2023-01-04T14:00:00Z') },
{ store: 'Store_B', product: 'Phone', quantity: 3, price: 700, salesValue: 2100, region: 'West', orderDate: ISODate('2023-01-05T15:00:00Z') },
{ store: 'Store_C', product: 'Tablet', quantity: 6, price: 300, salesValue: 1800, region: 'East', orderDate: ISODate('2023-01-06T16:00:00Z') }
]);
```

```
>_MONGOSH

> use storeDB
< switched to db storeDB
> db.createCollection('sales')
< { ok: 1 }
> db.sales.insertMany([
  { store: 'Store_A', product: 'Laptop', quantity: 10, price: 1500, salesValue: 15000, region: 'East', orderDate: ISODate('2023-01-01T10:00:00Z') },
  { store: 'Store_B', product: 'Phone', quantity: 5, price: 700, salesValue: 3500, region: 'West', orderDate: ISODate('2023-01-02T11:00:00Z') },
  { store: 'Store_C', product: 'Tablet', quantity: 7, price: 300, salesValue: 2100, region: 'East', orderDate: ISODate('2023-01-03T12:00:00Z') },
  { store: 'Store_A', product: 'Monitor', quantity: 4, price: 200, salesValue: 800, region: 'South', orderDate: ISODate('2023-01-04T14:00:00Z') },
  { store: 'Store_B', product: 'Phone', quantity: 3, price: 700, salesValue: 2100, region: 'West', orderDate: ISODate('2023-01-05T15:00:00Z') },
  { store: 'Store_C', product: 'Tablet', quantity: 6, price: 300, salesValue: 1800, region: 'East', orderDate: ISODate('2023-01-06T16:00:00Z') }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('672bbbe47c652a7bd87f75ec'),
      '1': ObjectId('672bbbe47c652a7bd87f75ed'),
      '2': ObjectId('672bbbe47c652a7bd87f75ee'),
      '3': ObjectId('672bbbe47c652a7bd87f75ef'),
      '4': ObjectId('672bbbe47c652a7bd87f75f0'),
      '5': ObjectId('672bbbe47c652a7bd87f75f1')
    }
  }
```

MAIN STAGES IN AGGREGATION PIPELINES

- Aggregation Pipeline consists of several stages, and each stage performs a specific operation on the data. Processes follow a specific order in the pipeline.

1. $MATCH

- Filter data based on certain conditions.
- Like the filtering operation in find or where in MongoDB.
- We start by filtering the data to display sales that pertain to the 'East' region.
- **db.sales.aggregate([{ $match: { region: "East" } }]);**

```
> db.sales.aggregate([
    { $match: { region: "East" } }
  ]);
< {
    _id: ObjectId('672bbbe47c652a7bd87f75ec'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z
  }
  {
    _id: ObjectId('672bbbe47c652a7bd87f75ee'),
    store: 'Store_C',
    product: 'Tablet',
    quantity: 7,
    price: 300,
    salesValue: 2100,
    region: 'East',
    orderDate: 2023-01-03T12:00:00.000Z
  }
```

2. $GROUP

- To group data based on a specific characteristic.
- Data is grouped according to a specific key, such as average or sum.
- We group sales by product and calculate the total sales of each product.
- db.sales.aggregate([{ $match: { region: "East" } }, { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }]);

```
> db.sales.aggregate([
    { $match: { region: "East" } },
    { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }
  ]);
< {
    _id: 'Laptop',
    totalSales: 15000
  }
  {
    _id: 'Tablet',
    totalSales: 3900
  }
```

3. $SORT

- Arranges documents according to a specific value.
- Data can be sorted in ascending or descending order based on the selected field.
- We sort the resulting collections by totalSales in descending order.
- db.sales.aggregate([{ $match: { region: "East" } }, { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }, { $sort: { totalSales: -1 } }]);

```
> db.sales.aggregate([
    { $match: { region: "East" } },
    { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } },
    { $sort: { totalSales: -1 } }
  ]);
< {
    _id: 'Laptop',
    totalSales: 15000
  }
  {
    _id: 'Tablet',
    totalSales: 3900
  }
```

4. $PROJECT

- Change the structure of documents.
- It is used to specify which fields should be included or hidden in the final results. You can also create new fields based on existing fields.
- We change the document structure to show only product with total sales

```
> db.sales.aggregate([
    { $match: { region: "East" } },
    { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } },
    { $sort: { totalSales: -1 } },
    { $project: { _id: 0, product: "$_id", totalSales: 1 } }
  ]);
< {
    totalSales: 15000,
    product: 'Laptop'
  }
  {
    totalSales: 3900,
    product: 'Tablet'
  }
```

5. $LIMIT

- Limit the number of documents displayed.
- Specifies the number of documents that will appear in the final result.
- We limit ourselves to displaying only the top 1 product based on total sales
- db.sales.aggregate([{ $match: { region: "East" } }, { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }, { $sort: { totalSales: -1 } }, { $limit: 1 }, { $project: { _id: 0, product: "$_id", totalSales: 1 } }]);

```
> db.sales.aggregate([
    { $match: { region: "East" } },
    { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } },
    { $sort: { totalSales: -1 } },
    { $limit: 1 },
    { $project: { _id: 0, product: "$_id", totalSales: 1 } }
  ]);
< {
    totalSales: 15000,
    product: 'Laptop'
  }
```

6. $SKIP

- Allows you to skip a certain number of documents in the result, useful for applications that require incremental loading of data.
- Usage: Typically used with skip and limit to create pagination.
- We skip the first product and then show the next products. Here we will display the products starting from the second product.
- db.sales.aggregate([{ $match: { region: "East" } }, { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }, { $sort: { totalSales: -1 } }, { $skip: 1 }, { $limit: 2 }, { $project: { _id: 0, product: "$_id", totalSales: 1 } }]);

```
> db.sales.aggregate([
    { $match: { region: "East" } },
    { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } },
    { $sort: { totalSales: -1 } },
    { $skip: 1 },
    { $limit: 2 },
    { $project: { _id: 0, product: "$_id", totalSales: 1 } }
  ]);
< {
    totalSales: 3900,
    product: 'Tablet'
  }
```

UPDATE DATA TO ADD ITEMS FIELD

```
// Update data in 'sales' array to add 'items' field as an array
db.sales.drop(); // Delete old data if it exists

db.sales.insertMany([
{
store: 'Store_A',
product: 'Laptop',
quantity: 10,
price: 1500,
salesValue: 15000,
region: 'East',
orderDate: ISODate('2023-01-01T10:00:00Z'),
items: [
{ productName: 'Laptop', quantity: 5, price: 1500 },
{ productName: 'Mouse', quantity: 5, price: 50 }
]
},
{
store: 'Store_B',
product: 'Phone',
quantity: 5,
price: 700,
salesValue: 3500,
region: 'West',
orderDate: ISODate('2023-01-02T11:00:00Z'),
items: [
{ productName: 'Phone', quantity: 3, price: 700 },
{ productName: 'Charger', quantity: 2, price: 50 }
]
},
```

```
{
store: 'Store_C',
product: 'tablet',
quantity: 7,
price: 300,
salesValue: 2100,
region: 'East',
orderDate: ISODate('2023-01-03T12:00:00Z'),
items: [
{ productName: 'Tablet', quantity: 5, price: 300 },
{ productName: 'Stylus', quantity: 2, price: 20 }
]
},
{
store: 'Store_A',
product: 'Monitor',
quantity: 4,
price: 200,
salesValue: 800,
region: 'South',
orderDate: ISODate('2023-01-04T14:00:00Z'),
items: [
{ productName: 'Monitor', quantity: 3, price: 200 },
{ productName: 'Stand', quantity: 1, price: 100 }
]
},
{
store: 'Store_B',
product: 'Phone',
quantity: 3,
price: 700,
salesValue: 2100,
region: 'West',
orderDate: ISODate('2023-01-05T15:00:00Z'),
items: [
{ productName: 'Phone', quantity: 2, price: 700 },
{ productName: 'Headset', quantity: 1, price: 100 }
]
},
{
store: 'Store_C',
product: 'tablet',
quantity: 6,
price: 300,
salesValue: 1800,
region: 'East',
orderDate: ISODate('2023-01-06T16:00:00Z'),
items: [
{ productName: 'Tablet', quantity: 4, price: 300 },
{ productName: 'Keyboard', quantity: 2, price: 50 }
]
}
]);
```

7. $UNWIND

- Convert arrays into multiple documents.
- Decomposes arrays so that each element in the array is represented as a separate document.
- Useful when the document contains arrays and you want to parse them as separate documents.
- Each order contains an array of items, and we want to unpack it.
- db.sales.aggregate([{ $unwind: "$items" }]);

```
> db.sales.aggregate([
    { $unwind: "$items" }
  ]);
< {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z,
    items: {
      productName: 'Laptop',
      quantity: 5,
      price: 1500
    }
  }
  {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z,
    items: {
      productName: 'Mouse',
      quantity: 5,
      price: 50
    }
```

8. $ADDFIELDS

- Add or modify fields in documents
- We add a new field called totalCost that calculates the total cost by multiplying the quantity by the price.
- db.sales.aggregate([{ $addFields: { totalCost: { $multiply: ["$quantity", "$price"] } } }]);

```
> db.sales.aggregate([
    { $addFields: { totalCost: { $multiply: ["$quantity", "$price"] } } }
  ]);
< {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z,
    items: [
      {
        productName: 'Laptop',
        quantity: 5,
        price: 1500
      },
      {
        productName: 'Mouse',
        quantity: 5,
        price: 50
      }
    ],
    totalCost: 15000
  }
```

CREATE A PRODUCTS COLLECTION

- db.products.insertMany([{ product_name: "Laptop", price: 1500 }, { product_name: "Phone", price: 700 }, { product_name: "Tablet", price: 300 }, { product_name: "Monitor", price: 200 }]);

```
> db.products.insertMany([
    { product_name: "Laptop", price: 1500 },
    { product_name: "Phone", price: 700 },
    { product_name: "Tablet", price: 300 },
    { product_name: "Monitor", price: 200 }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('672bc4557c652a7bd87f75f8'),
      '1': ObjectId('672bc4557c652a7bd87f75f9'),
      '2': ObjectId('672bc4557c652a7bd87f75fa'),
      '3': ObjectId('672bc4557c652a7bd87f75fb')
    }
  }
```

9. $LOOKUP

- Join another data set.
- Allows joining other groups within the same database. This operator is used to perform operations similar to JOIN in SQL.
- We use the $lookup operation to associate products data with sales data based on product in the sales collection and product_name in the products collection.

db.**sales**.aggregate([{
$lookup: {
from: 'products', // The name of the group we want to link (products)
localField: 'product', // The field in the sales group that we use for matching
foreignField: 'product_name', // The field in the products collection that we use for matching
as: 'product_details' // The name we use to save the merged data from the products}}]);

```
< {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z,
    items: [
      {
        productName: 'Laptop',
        quantity: 5,
        price: 1500
      },
      {
        productName: 'Mouse',
        quantity: 5,
        price: 50
      }
    ],
    product_details: [
      {
        _id: ObjectId('672bc4557c652a7bd87f75f8'),
        product_name: 'Laptop',
        price: 1500
      }
    ]
  }
```

```
> db.sales.aggregate([
   { $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } },
   { $sort: { totalSales: -1 } },
   { $merge: { into: "sales_summary" } }
 ]);
```

localhost:27017 > storeDB > sales_summary

Documents 4 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

ADD DATA | EXPORT DATA | UPDATE | DELETE

```
_id: "Laptop"
totalSales : 15000

_id: "Phone"
totalSales : 5600

_id: "Tablet"
totalSales : 3900

_id: "Monitor"
totalSales : 800
```

10. $MERGE

- Merge data into a new group or update an existing group.
- We store the final results in a new collection named sales_summary
- db.sales.aggregate([{ $group: { _id: "$product", totalSales: { $sum: "$salesValue" } } }, { $sort: { totalSales: -1 } }, { $merge: { into: "sales_summary" } }]);
- whenMatched option : Optional. The behavior of $merge if a result document and an existing document in the collection have the same value for the specified on field(s). See Actions here : https://www.mongodb.com/docs/manual/reference/operator/aggregation/merge/
- whenNotMatched option : Optional. The behavior of $merge if a result document does not match an existing document in the out collection. See Actions here : https://www.mongodb.com/docs/manual/reference/operator/aggregation/merge/

11. $SAMPLE

- It is used to select a random sample of documents from the collection
- select a random sample of 3 documents from sales

```
db.sales.aggregate([
  { $sample: { size: 3 } }
]);
```

```
> db.sales.aggregate([
    { $sample: { size: 3 } }
  ]);
< {
    _id: ObjectId('672bc12c7c652a7bd87f75f3'),
    store: 'Store_B',
    product: 'Phone',
    quantity: 5,
    price: 700,
    salesValue: 3500,
    region: 'West',
    orderDate: 2023-01-02T11:00:00.000Z,
    items: [
      {
        productName: 'Phone',
        quantity: 3,
        price: 700
      },
      {
        productName: 'Charger',
        quantity: 2,
        price: 50
      }
    ]
  }
  {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
```

12. $COUNT

- It is used to count the number of documents in the collection and return only one result.
- Count the total number of sales in the sales group.
- db.sales.aggregate([{ $count: "total_sales" }]);

```
> db.sales.aggregate([
    { $count: "total_sales" }
  ]);
< {
    total_sales: 6
  }
```

```
> db.sales.aggregate([
    { $sortByCount: "$product" }
  ]);
< {
    _id: 'Phone',
    count: 2
  }
  {
    _id: 'Tablet',
    count: 2
  }
  {
    _id: 'Monitor',
    count: 1
  }
  {
    _id: 'Laptop',
    count: 1
  }
```

13. $SORTBYCOUNT

- Used to count the number of times a particular value appears, and its order.
- Count the number of times each product appears in sales.

```
db.sales.aggregate([
  { $sortByCount: "$product" }
]);
```

```
> db.sales.aggregate([
    {
      $set: {
        total_cost: { $multiply: ["$quantity", "$price"] }
      }
    }
  ]);
< {
    _id: ObjectId('672bc12c7c652a7bd87f75f2'),
    store: 'Store_A',
    product: 'Laptop',
    quantity: 10,
    price: 1500,
    salesValue: 15000,
    region: 'East',
    orderDate: 2023-01-01T10:00:00.000Z,
    items: [
      {
        productName: 'Laptop',
        quantity: 5,
        price: 1500
      },
      {
        productName: 'Mouse',
        quantity: 5,
        price: 50
      }
    ],
    total_cost: 15000
  }
```

14. $SET

- Used to add a new field or update an existing field in documents.
- Add a new field called total_cost that calculates the total value based on quantity and price

```
db.sales.aggregate([
  {
    $set: {
      total_cost: { $multiply: ["$quantity", "$price"] }
    }
  }
]);
```

15. $BUCKETAUTO

SPLIT VALUES INTO THREE AUTOMATIC CATEGORIES (PRICE).

```
> db.sales.aggregate([
    {
      $bucketAuto: {
        groupBy: "$price",
        buckets: 3
      }
    }
  ]);
< {
    _id: {
      min: 200,
      max: 700
    },
    count: 3
  }
  {
    _id: {
      min: 700,
      max: 1500
    },
    count: 2
  }
  {
    _id: {
      min: 1500,
      max: 1500
    },
    count: 1
  }
```

WHAT IS MONGODB COMPASS?

- MongoDB Compass is a powerful GUI for querying, aggregating, and analyzing your MongoDB data in a visual environment.
- The Compass Home screen includes the following sections:
  - ✓ Connections Sidebar
  - ✓ Your saved queries and aggregation pipelines.
- Once connected to a MongoDB instance, you can reach the Compass Home screen by clicking a cluster name in the Connections Sidebar.
- The Connections Sidebar on the left of the Compass window contains a list of your favorite and saved connections. Active connections have a green dot next to their name.

MongoDB Compass

Connections Edit View Help

Compass

My Queries

CONNECTIONS (5)

Search connections

- localhost:27017
  - admin
  - config
  - local
  - mycodeworks
  - resturants
  - sample_mflix
  - storeDB
  - students
- localhost:27017
- localhost:27017
- localhost:27017
- localhost:27017

MANAGE DATABASES IN COMPASS

- A database is a container for collections. Each database gets its own set of files on the host file system. A single MongoDB server typically has multiple databases.

- The Databases tab lists the existing databases for your MongoDB deployment. To access the Databases tab, click the deployment name in the Connections Sidebar.
- From this view, you can click a database name in the sidebar to view its collections. Alternatively, you can view database collections by clicking the desired database in the left-hand navigation.
- You can also create or drop databases from this view.

MongoDB Compass - localhost:27017/storeDB

Connections Edit View Help

Compass

My Queries

CONNECTIONS (5)

Search connections

- localhost:27017
  - admin
  - config
  - local
  - mycodeworks
  - resturants
  - sample_mflix
  - storeDB
    - products
    - sales
  - students
- localhost:27017
- localhost:27017
- localhost:27017
- localhost:27017

storeDB

localhost:27017 > storeDB

Open MongoDB shell | Create collection | Refresh

Sort by Collection Name

View

products

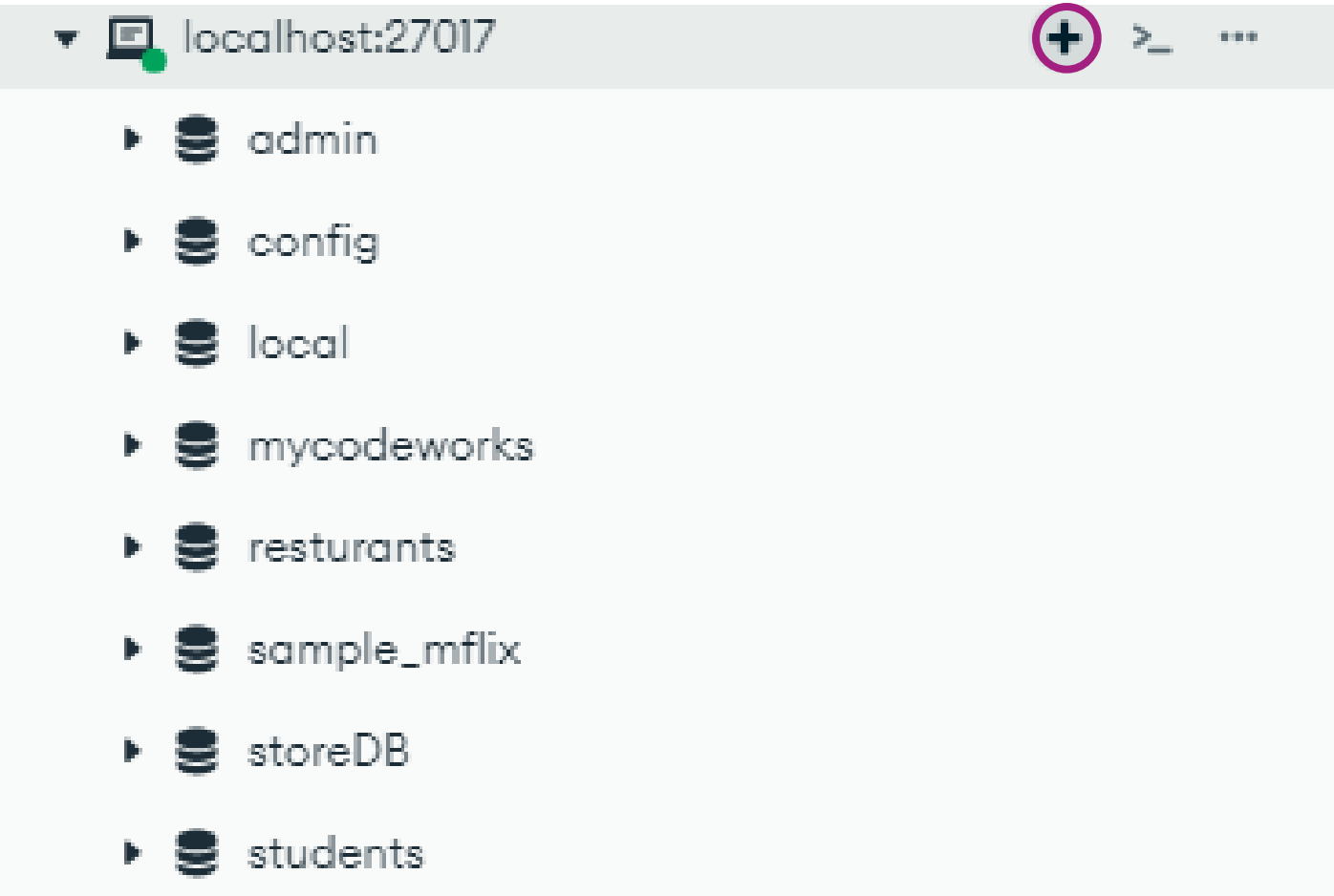
Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
20.48 kB	4	58.00 B	1	20.48 kB

sales

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
20.48 kB	6	264.00 B	1	20.48 kB

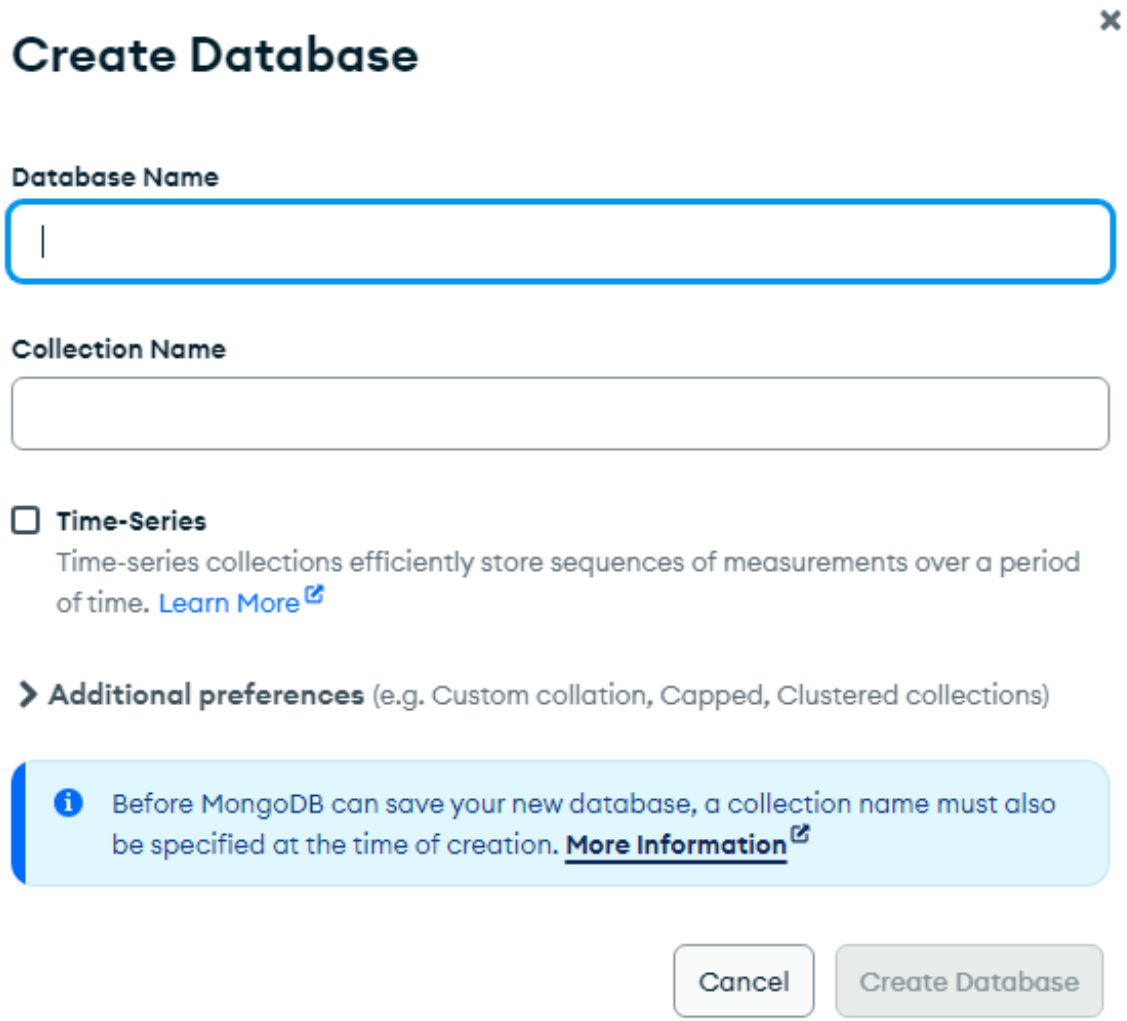
CREATE A DATABASE

1. Open the Create Database dialog : In the Connections Sidebar, click the icon to the right of the connection name to bring up the Create Database dialog.



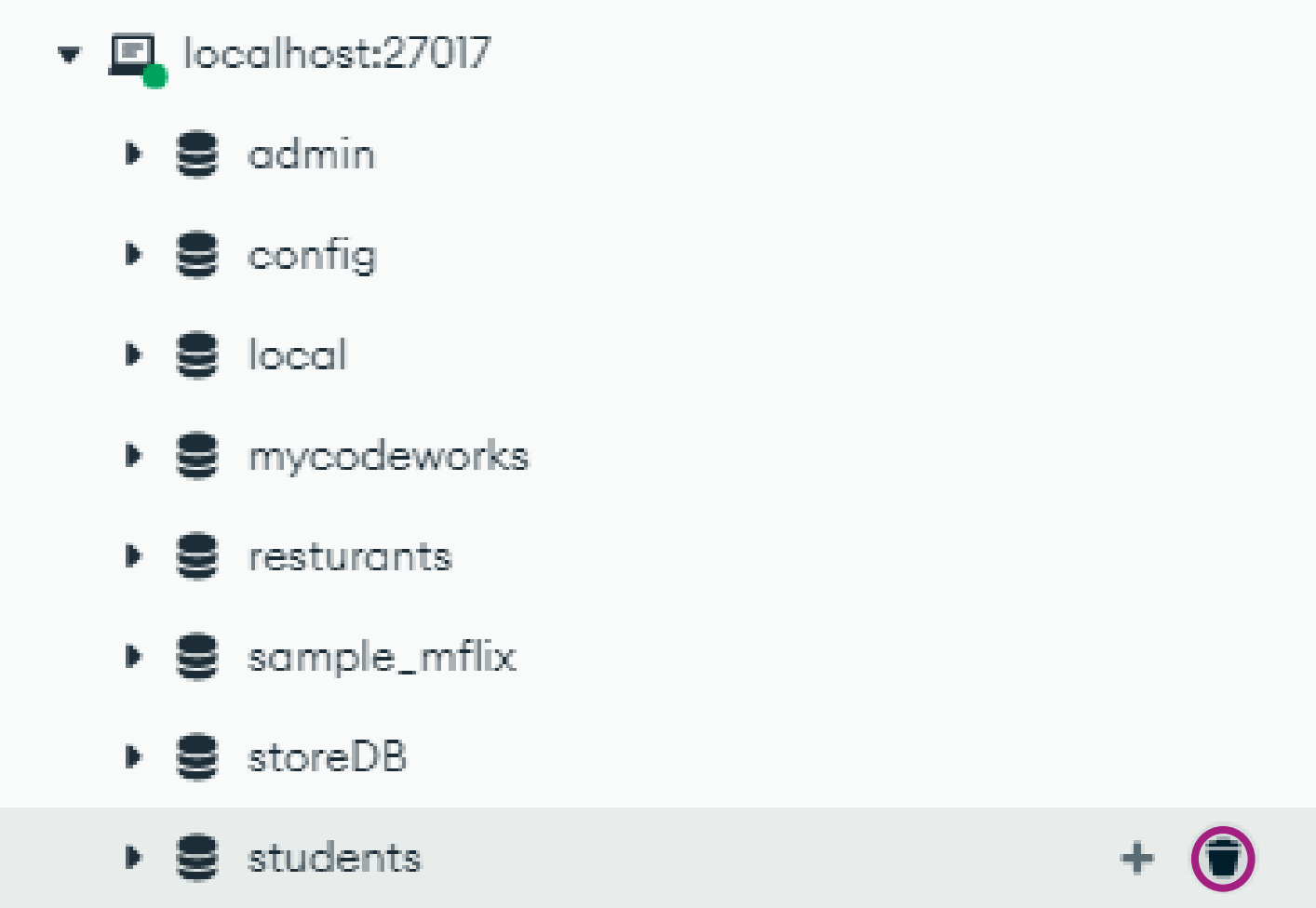
CREATE A DATABASE

2. Enter database and first collection information: In the dialog, enter the name of the database to create and its first collection. Both the database name and the collection name are required.
3. Click Create Database to create the database and its first collection.

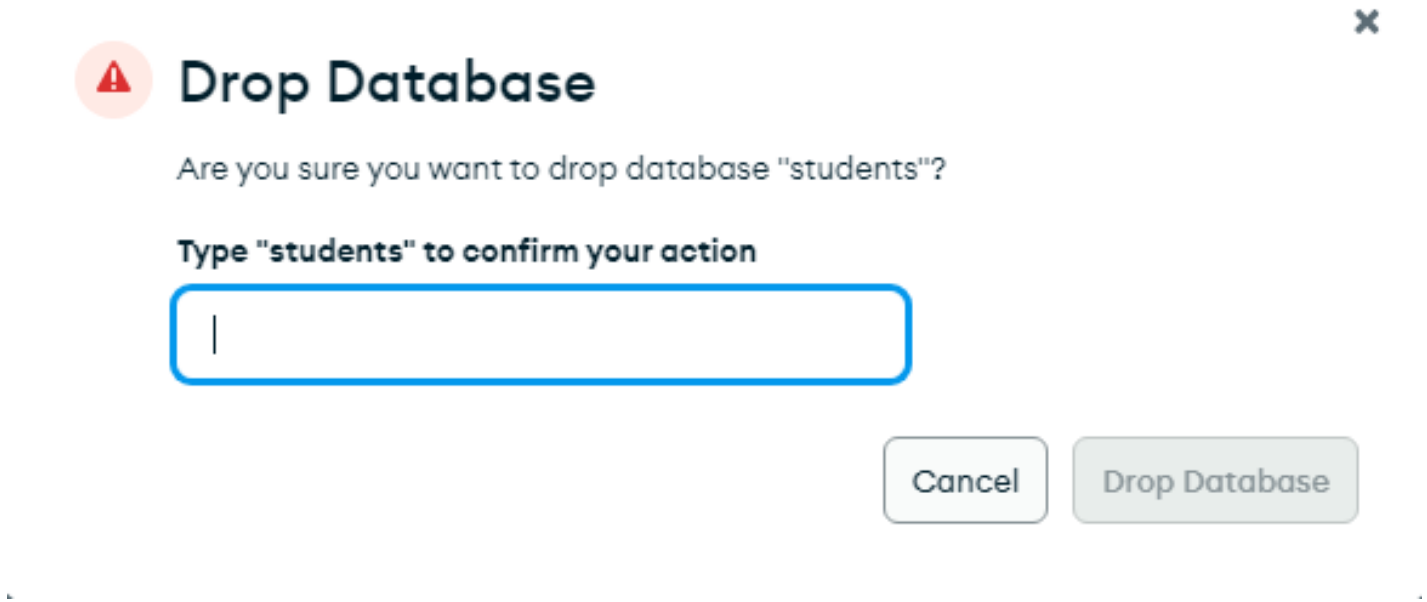


DROP A DATABASE

1. Click the trash can icon for the database: From the Databases tab, to delete a database, click on the trash can icon for that database. A confirmation dialog appears.



DROP A DATABASE



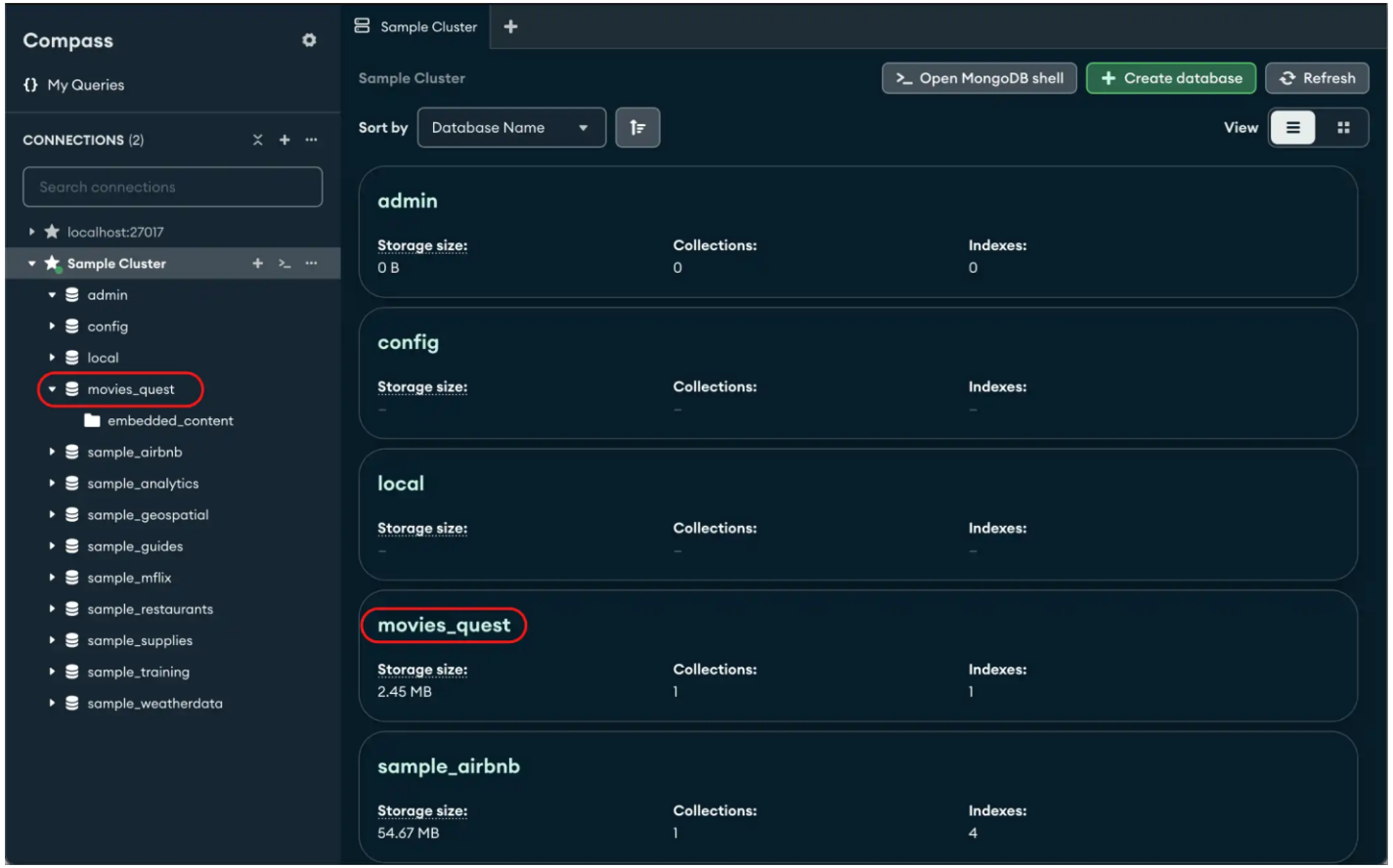
2. Confirm the database to delete: In the dialog, enter the name of the database to delete.
3. Click Drop Database to delete the database.

MANAGE COLLECTIONS IN COMPASS

- A collection is a grouping of MongoDB documents. Documents within a collection can have different fields. A collection is the equivalent of a table in a relational database system. A collection exists within a single database.

COLLECTIONS SCREEN

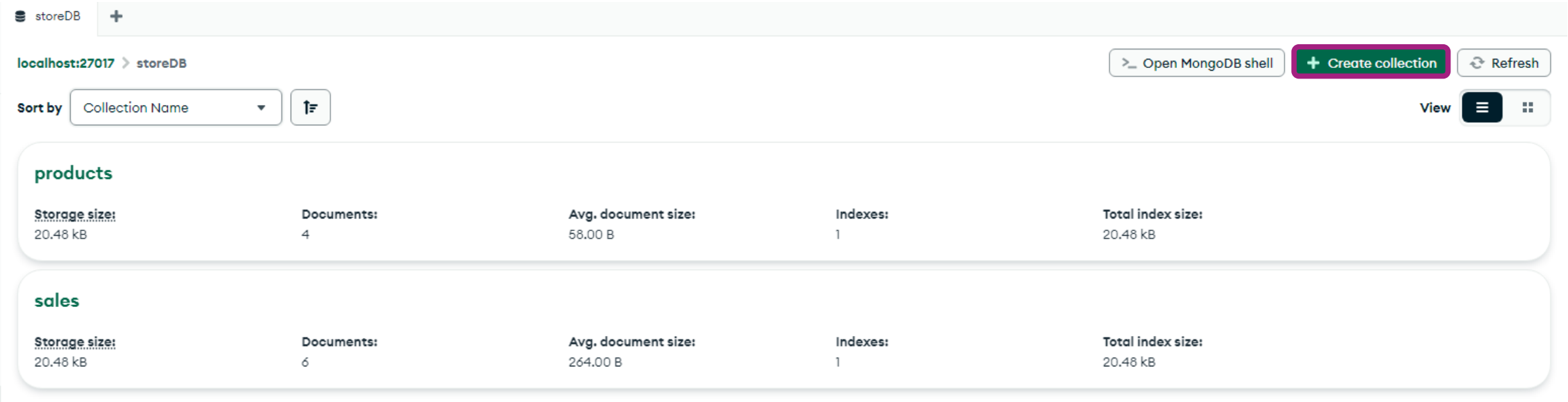
- The Collections screen lists the existing collections and views in the selected database. Each list item includes the name and other general information for the collection or view.
- To access the Collections screen for a database, from the Databases screen either:
  - Click a Database Name in the main Databases view, or
  - Click a database in the left navigation.



- The Collections screen displays the following information for each collection in the selected database:
  - ✓ Collection name
  - ✓ Number of documents in the collection
  - ✓ Average size of documents in the collection
  - ✓ Total size of all documents in the collection
  - ✓ Number of indexes on the collection
  - ✓ Total size of all indexes on the collection
  - ✓ Collation properties for the collection. Hover over a Collation banner to view the properties for that collection.

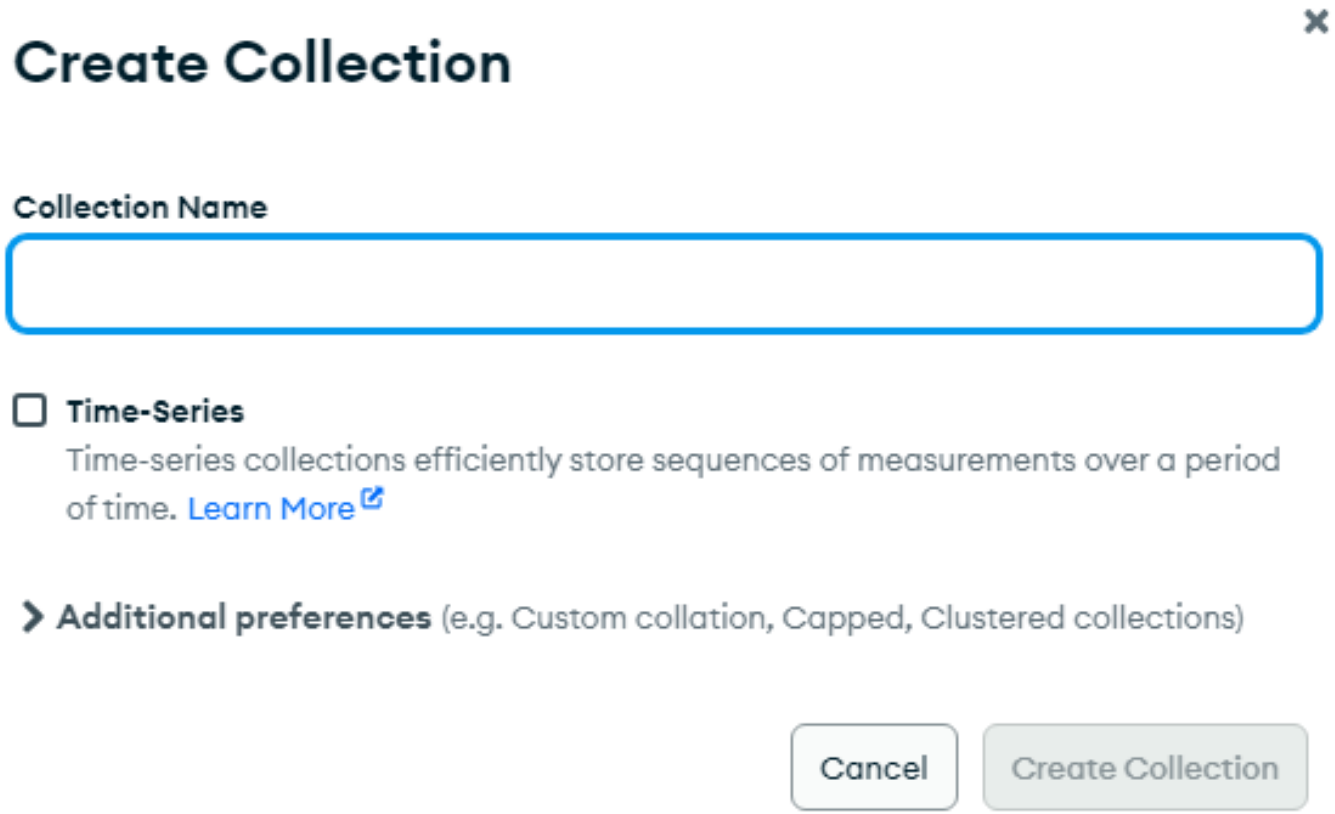
CREATE A COLLECTION

1. Click the Create Collection button: From the Collections screen, click the Create Collection button.



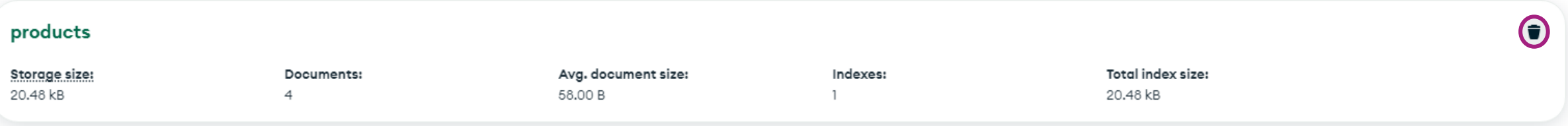
CREATE A COLLECTION

2. Enter the collection information: In the Create Collection dialog, enter the name of the collection to create.
3. Click Create Collection to create the collection.

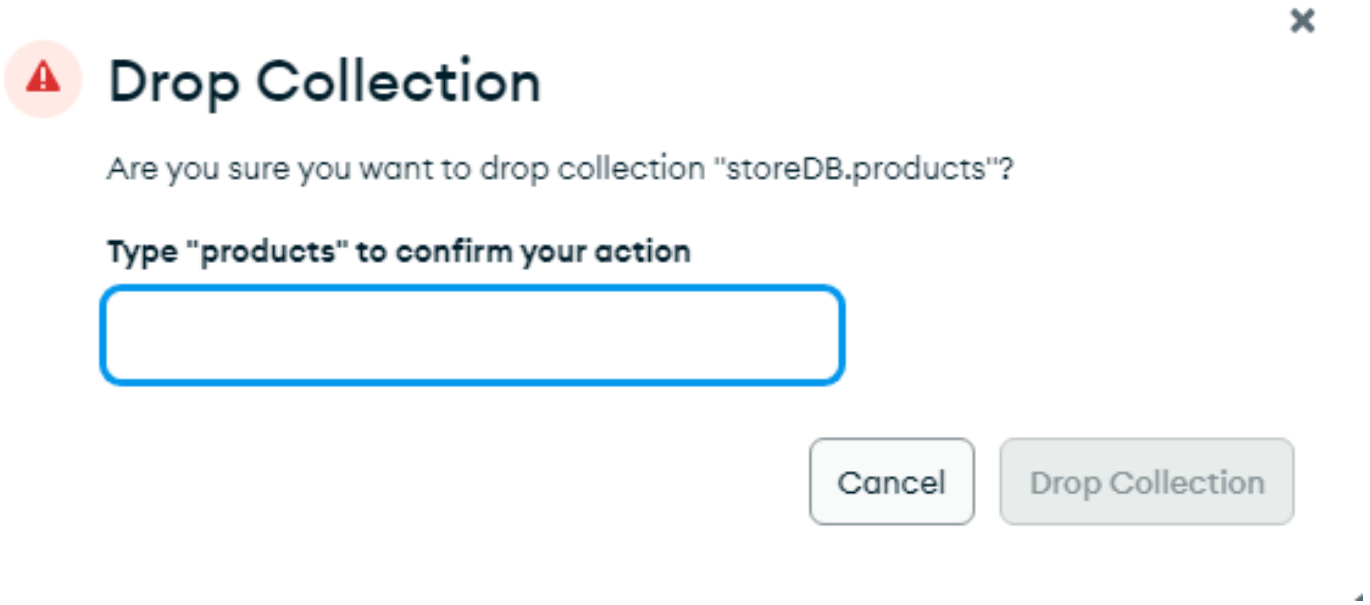


DROP A COLLECTION

1. Click the trash can icon to delete a collection: From the Collections screen, click on the trash can for the collection to delete. A confirmation dialog appears.



DROP A COLLECTION



2. Confirm the collection to delete: In the dialog, enter the name of the collection to delete.
3. Click Drop Collection to drop the collection.

- The Collection Detail screen shows detailed information for a collection, including the documents the collection contains. To see collection details, either:
  - ✓ Click a Collection Name in the main Collections screen, or
  - ✓ Click a collection in the left navigation.
  - ✓ Open the collection in a new tab.
- After you select a collection, Compass shows you that collection's Documents tab. Compass provides the following collection information and functionality in the detailed view:
  - ✓ Manage Documents in Compass
  - ✓ Import and Export Data to and from the collection
  - ✓ Create an Aggregation Pipeline
  - ✓ Schema Analysis
  - ✓ View Query Performance
  - ✓ Manage Indexes
  - ✓ Set Validation Rules for Your Schema

storeDB | products

localhost:27017 > storeDB > products

Open MongoDB shell

Documents 4 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | EXPORT DATA | UPDATE | DELETE

25 | 1 - 4 of 4

```
_id: ObjectId('673478a8d0c12829c73fcf55')
product_name : "Laptop"
price : 1500
```

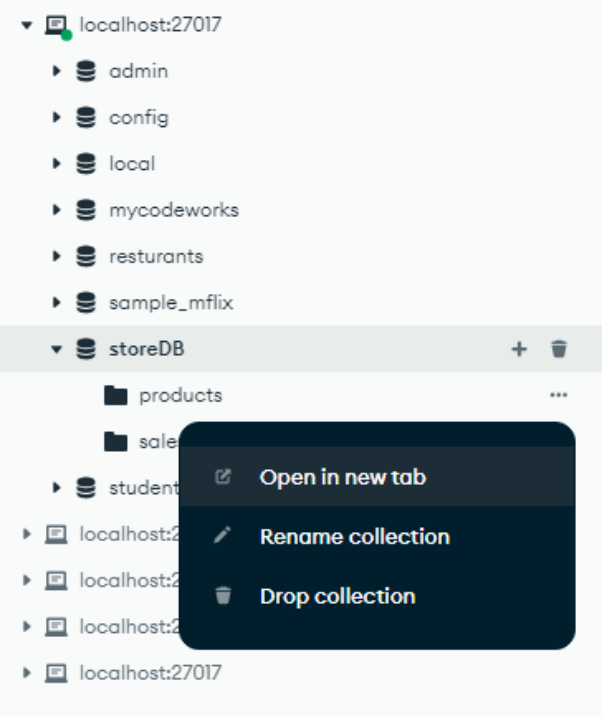
```
_id: ObjectId('673478a8d0c12829c73fcf56')
product_name : "Phone"
price : 700
```

```
_id: ObjectId('673478a8d0c12829c73fcf57')
product_name : "Tablet"
price : 300
```

```
_id: ObjectId('673478a8d0c12829c73fcf58')
product_name : "Monitor"
price : 200
```

TABBED VIEW

- You can open multiple Collection Detail screens in separate tabs. To open a new tab to view collection details:
  1. Click the caret icon next to the database which contains the collection you wish to view.
  2. Hover over the desired collection.
  3. Click the appearing ellipses (...) button.
  4. Click Open in New Tab.

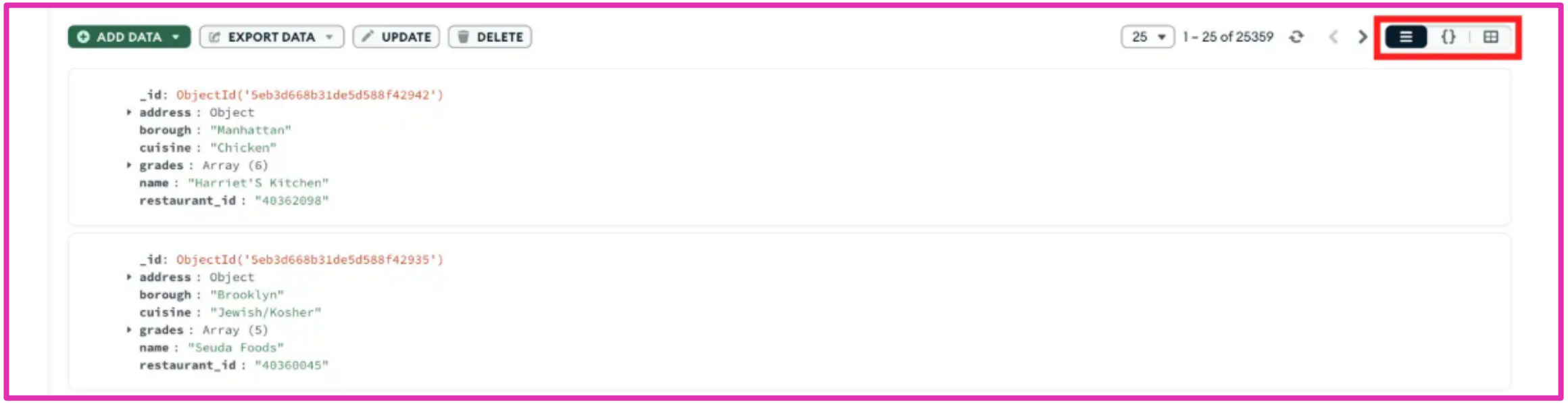


MANAGE DOCUMENTS IN COMPASS

- Documents are individual records in a MongoDB collection and are the basic unit of data in MongoDB.
- From the Documents tab, you can view, insert, modify, modify multiple, clone, delete, and delete multiple documents in your selected collection or view.
- Note: For query result sets larger than 20 documents, Compass shows paginated results. By default, pages display 20 documents at a time.
- From the query bar, you can specify a query to filter the displayed documents. Click Options to specify query options.

- The Documents tab provides three ways to access documents:
  1. List View: The default document view. Documents are shown as individual members of a list. In this view you can easily expand embedded objects and arrays.
  2. JSON View : Documents are shown as properly-formatted JSON objects. In this view Compass uses extended JSON to display the data types of fields where appropriate.
  3. Table View : Shows documents as rows of a table, with document fields shown in the table columns. With this view, you can more easily see which documents contain specific field values.
- Use the View buttons to select which view is shown in the following examples:

LIST VIEW


ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1 - 25 of 25359
_id: ObjectId('5eb3d668b31de5d588f42942')
address : Object
borough : "Manhattan"
cuisine : "Chicken"
grades : Array (6)
name : "Harriet'S Kitchen"
restaurant_id : "40362098"
_id: ObjectId('5eb3d668b31de5d588f42935')
address : Object
borough : "Brooklyn"
cuisine : "Jewish/Kosher"
grades : Array (5)
name : "Seuda Foods"
restaurant_id : "40360045"

JSON VIEW

ADD DATA | EXPORT DATA | UPDATE | DELETE | 25 | 1 - 25 of 25359


```
{
  "_id": {…},
  "address": {…},
  "borough": "Manhattan",
  "cuisine": "Chicken",
  "grades": […],
  "name": "Harriet'S Kitchen",
  "restaurant_id": "40362098"
}
{
  "_id": {…},
  "address": {…},
  "borough": "Brooklyn",
  "cuisine": "Jewish/Kosher",
  "grades": […],
  "name": "Seuda Foods",
  "restaurant_id": "40360045"
}
{
  "_id": {…},
  "address": {…},
  "borough": "Manhattan",
  "cuisine": "Italian",
  "grades": […],
  "name": "Angelo Of Mulberry St.",
  "restaurant_id": "40365293"
}
```

TABLE VIEW

ADD DATA | EXPORT DATA | UPDATE | DELETE

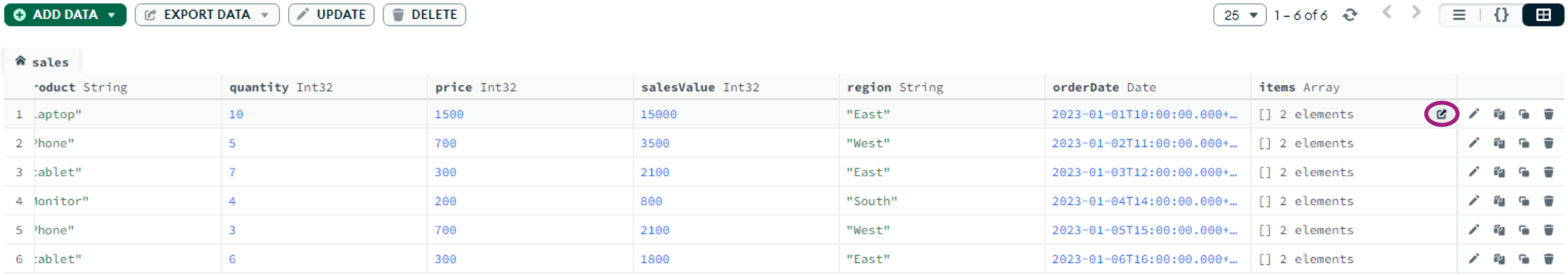
25 | 1 – 25 of 25359

restaurants

	_id ObjectId	address Object	borough String	cuisine String	grades Array	name String	restaura
1	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Manhattan"	"Chicken"	[] 6 elements	"Harriet'S Kitchen"	"4036209
2	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Brooklyn"	"Jewish/Kosher"	[] 5 elements	"Seuda Foods"	"4036004
3	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Manhattan"	"Italian"	[] 5 elements	"Angelo Of Mulberry St."	"4036529
4	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Staten Island"	"American"	[] 5 elements	"Richmond County Country …	"4036692
5	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Brooklyn"	"Pizza/Italian"	[] 5 elements	"Da Vinci Pizza"	"4036708
6	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Manhattan"	"Irish"	[] 5 elements	"Twins Pub"	"4036717
7	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Manhattan"	"Irish"	[] 4 elements	"Rosie O'Grady Restaurant"	"4037471
8	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Queens"	"American"	[] 5 elements	"Europa Go-Go Dancing"	"4037481
9	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Bronx"	"Chinese/Cuban"	[] 3 elements	"Sabrosura Restaurant"	"4037513
10	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Brooklyn"	"American"	[] 6 elements	"Vegas Diner"	"4037597
11	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Manhattan"	"Mexican"	[] 6 elements	"Mary Ann'S"	"4037827
12	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Queens"	"Café/Coffee/Tea"	[] 4 elements	"Steve'S Coffee Shop"	"4036905
13	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Queens"	"Greek"	[] 5 elements	"Fontana Famous Pizza & G…	"4037607
14	ObjectId('5eb3d668b31de5d…	{} 4 fields	"Bronx"	"Seafood"	[] 5 elements	"Jp'S Waterside Restauran…	"4037712

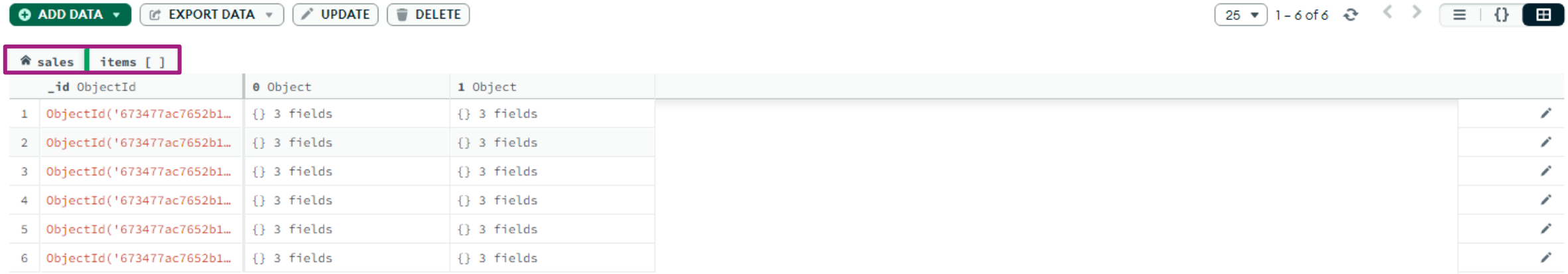
EXPAND EMBEDDED OBJECTS AND ARRAYS

- To view nested object fields and array elements, hover your cursor over a field with a value type of Object or Array and click the button with outward-pointing arrows which appears on the right side of the field.
- Compass opens a new tab in the Table View corresponding to the nested element.



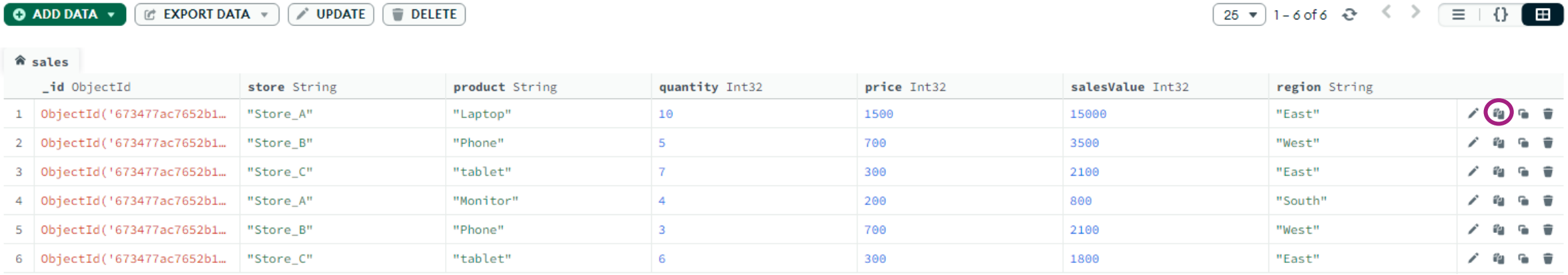
	roduct String	quantity Int32	price Int32	salesValue Int32	region String	orderDate Date	items Array
1	.aptop"	10	1500	15000	"East"	2023-01-01T10:00:00.000+…	[] 2 elements
2	'hone"	5	700	3500	"West"	2023-01-02T11:00:00.000+…	[] 2 elements
3	:ablet"	7	300	2100	"East"	2023-01-03T12:00:00.000+…	[] 2 elements
4	lonitor"	4	200	800	"South"	2023-01-04T14:00:00.000+…	[] 2 elements
5	'hone"	3	700	2100	"West"	2023-01-05T15:00:00.000+…	[] 2 elements
6	:ablet"	6	300	1800	"East"	2023-01-06T16:00:00.000+…	[] 2 elements

- The following example displays the items object nested within the sales collection documents. The new tab displays the properties of the items object:



COPY DOCUMENTS TO CLIPBOARD

- To copy a document to your clipboard, hover over the document and click the Copy icon:



	_id ObjectId	store String	product String	quantity Int32	price Int32	salesValue Int32	region String
1	ObjectId('673477ac7652b1…	"Store_A"	"Laptop"	10	1500	15000	"East"
2	ObjectId('673477ac7652b1…	"Store_B"	"Phone"	5	700	3500	"West"
3	ObjectId('673477ac7652b1…	"Store_C"	"tablet"	7	300	2100	"East"
4	ObjectId('673477ac7652b1…	"Store_A"	"Monitor"	4	200	800	"South"
5	ObjectId('673477ac7652b1…	"Store_B"	"Phone"	3	700	2100	"West"
6	ObjectId('673477ac7652b1…	"Store_C"	"tablet"	6	300	1800	"East"

PRACTICE

1. Create Database named as “CompanyDB”.
2. Insert the following data in collection with name “employees”:

   { name: "Ahmed", salary: 5000, department: "IT" },

   { name: "Mona", salary: 7000, department: "HR" },

   { name: "Ali", salary: 4500, department: "Finance" },

   { name: "Sara", salary: 6000, department: "IT" }
3. Increase the salaries of all employees in the IT section by 10%.
4. Using Aggregation Pipelines :
   a) Filter employees whose salary is more than 5000 using $match.
   b) Calculate the number of employees in each department using $group.
   c) Divide salaries into 3 categories using $bucketAuto.
   d) Choose 2 employees randomly $sample.

SOLUTION

1. use CompanyDB
2. db.employees.insertMany([{ name: "Ahmed", salary: 5000, department: "IT" },
   { name: "Mona", salary: 7000, department: "HR" },
   { name: "Ali", salary: 4500, department: "Finance" },
   { name: "Sara", salary: 6000, department: "IT" }])
3. db.employees.updateMany(
   { department: "IT" },
   { $mul: { salary: 1.1 } })
4. Aggregation Pipelines:
   a. db.employees.aggregate([{ $match: { salary: { $gt: 5000 } } }])
   b. db.employees.aggregate([{ $group: { _id: "$department", count: { $sum: 1 } } }])
   c. db.employees.aggregate([{ $bucketAuto: { groupBy: "$salary", buckets: 3 } }])
   d. db.employees.aggregate([{ $sample: { size: 2 } }])

THANK YOU